

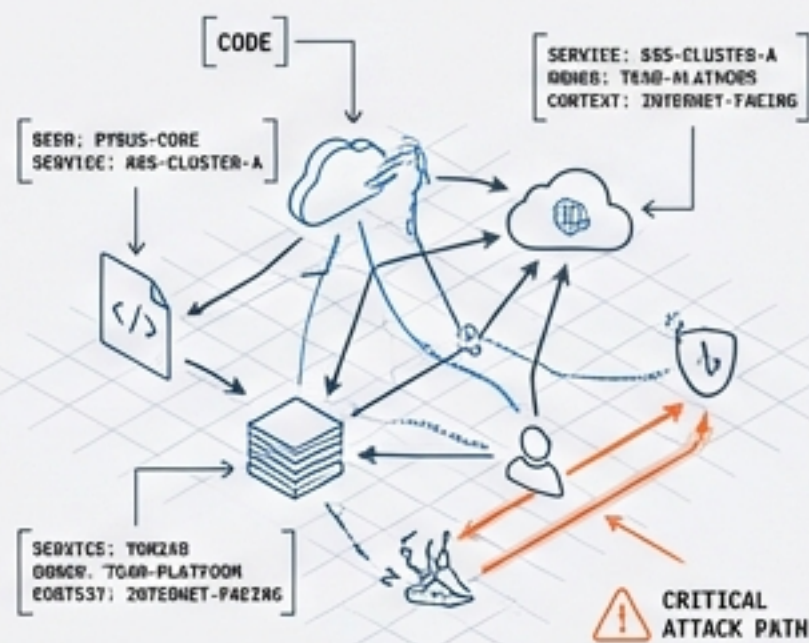
The Core Differentiator: Actionable ASPM via Connectivity

Architectural Analysis: Moving Beyond Lists to “Code-to-Cloud” Relationships

NARRATIVE: FROM ISOLATION TO CONNECTION

Phoenix Security distinguishes itself by moving beyond vulnerability listing to a “Code-to-Cloud” relationship model.

The central value proposition is the **Vulnerability Intelligence Graph**, which transforms isolated findings into a connected map of ownership, runtime context, and threat threat intelligence.



Analyst Note: This briefing explicitly distinguishes between **Evidence** (Public Documentation/APIs) and **Inference** (Architectural Deductions).



SCOPE OF ANALYSIS

- 1 Technology Surface Area**
Evidenced by documentation, PYRUS repos, and product releases.
- 2 Graph Data Models**
Analysis of the Risk, Organisational, and Intelligence graph layers.
- 3 GenAI Integration**
How the graph acts as the substrate for Agentic workflows and MCP.



The Graph Philosophy: Three Dimensions of Context

Architectural Analysis: Unifying Asset, Organisational, and Threat Intelligence for Holistic Security Context

Layer 1: The Cyber Risk Graph (Asset View)

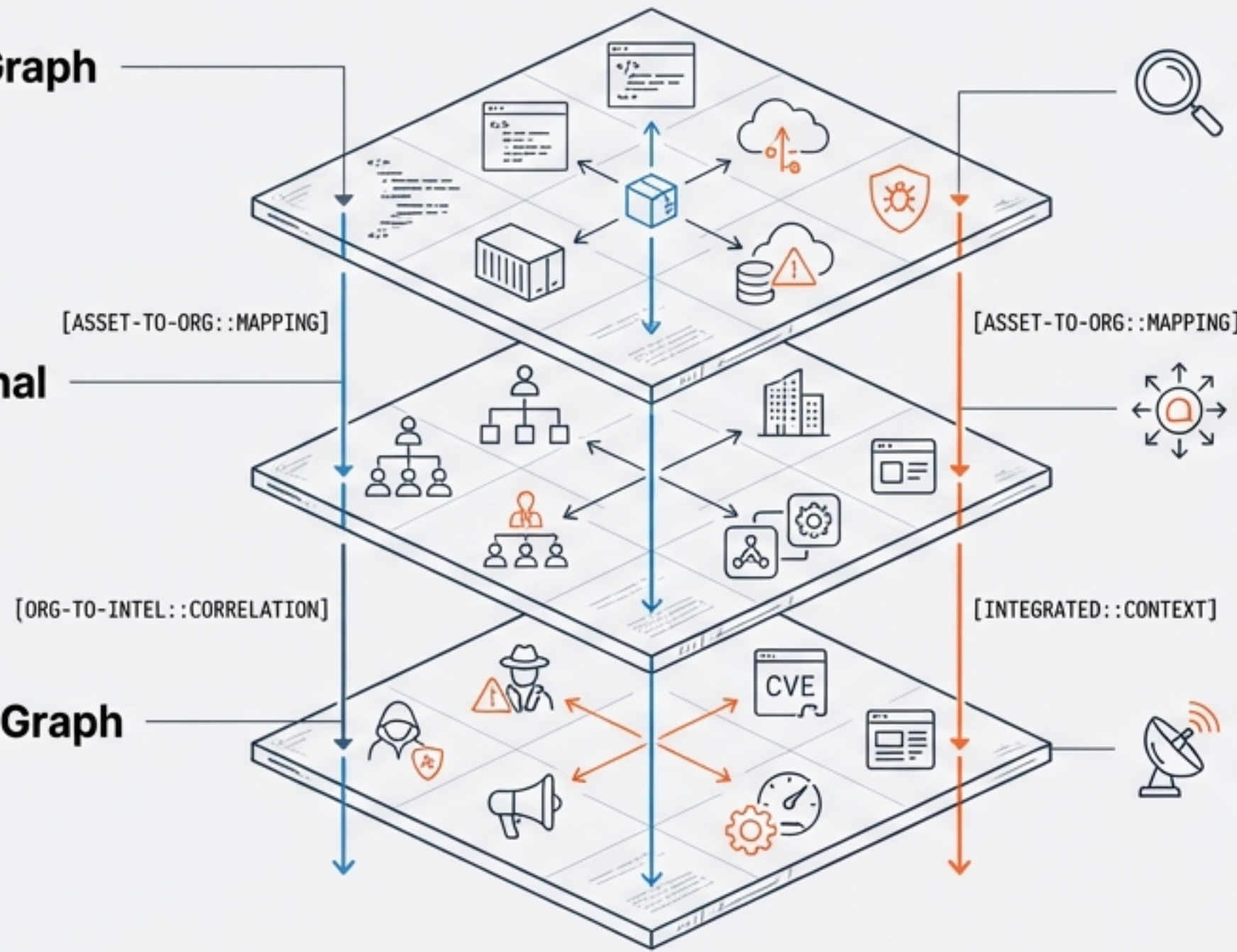
[LAYER-01::ASSET-VIEW]

Layer 2: The Organisational Graph (Business View)

[LAYER-02::ORG-VIEW]

Layer 3: The Intelligence Graph (External View)

[LAYER-03::INTEL-VIEW]



Connecting findings to assets

Correlating static code analysis (SAST) with runtime environments and infrastructure.

[DATA-FLOW::SAST->RUNTIME]



Blast Radius Analysis

Mapping vulnerabilities to Business Units, Teams, and Applications. Answering: "Who owns this?" and assessing potential impact on business functions.

[INPACT::OWNERSHIP+CRITICALITY]



Threat Context

Linking internal findings to external Threat Actors, Campaigns, and Exploitability metrics. Prioritizing based on real-world threat landscape.

[CONTEXT::THREAT+EXPLOITABILITY]



Phoenix creates a single connected model to reduce impedance for traversal-heavy queries, enabling seamless navigation across all three dimensions of context for faster, more accurate decision-making.

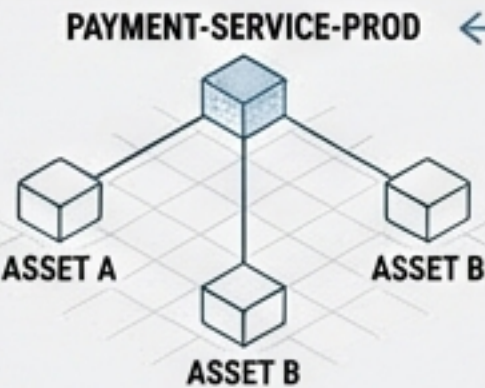
Data Ingestion & Modelling: The PYRUS Framework

A YAML-native automation engine for a 'Living CMDB'.

THE CODE

YAML CONFIG SNIPPET: AssetDefinition

```
apiVersion: phoenix/v1
kind: AssetDefinition
metadata:
  name: "payment-service-prod"
  owner: "team-fintech-core"
spec:
  assetType: "CONTAINER"
  environment: "AWS_PROD_EU_WEST"
  repository: "github.com/org/pay-svc"
```

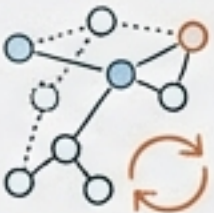


THE FEATURES



YAML-Native Automation

Configures assets via code, integrating with CI/CD and Identity Providers.



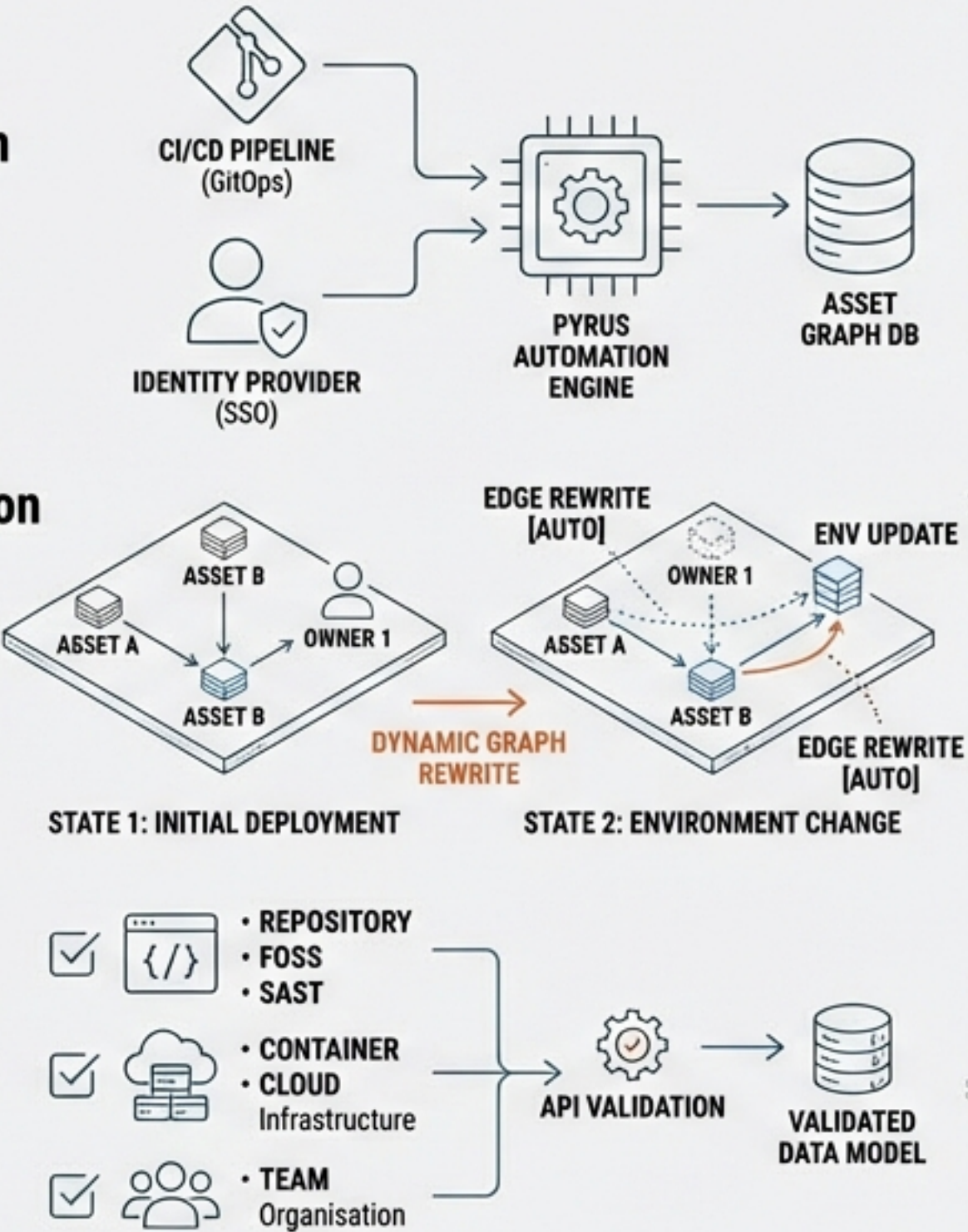
Self-Healing Configuration

Auto-groups assets by service and owner. When environments change, the graph rewrites edges automatically.



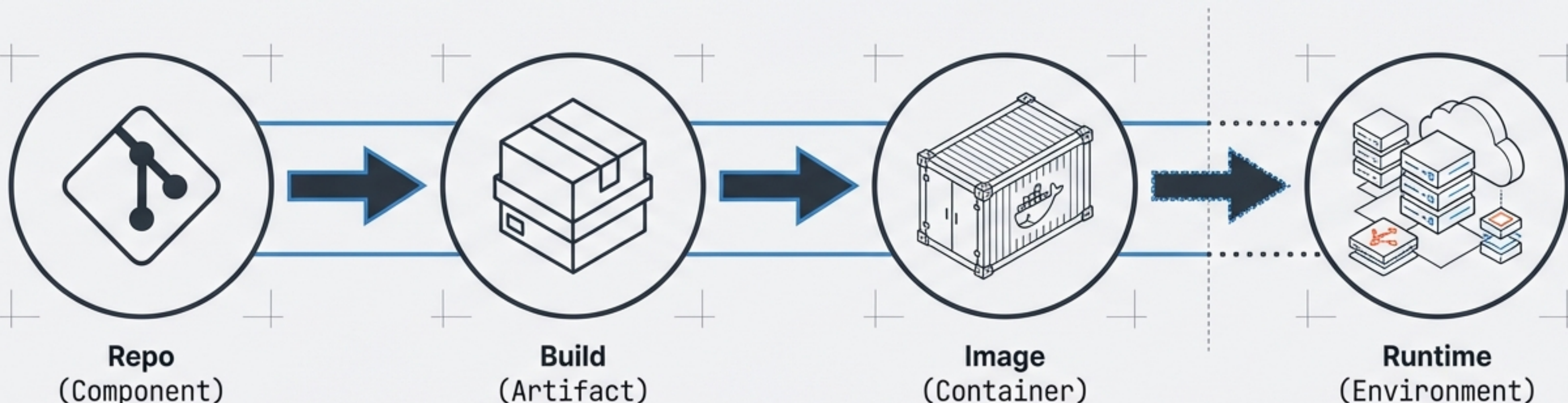
Explicit Taxonomy

API validates specific AssetType values proving multi-domain design:



Code-to-Cloud Lineage & Reachability

Contextual Deduplication via Connectivity.



Reachability Analysis

Distinguishes between static presence and runtime execution to deprioritise inactive risks.



Throttling

Uses lineage to reduce alert fatigue by excluding inactive images.

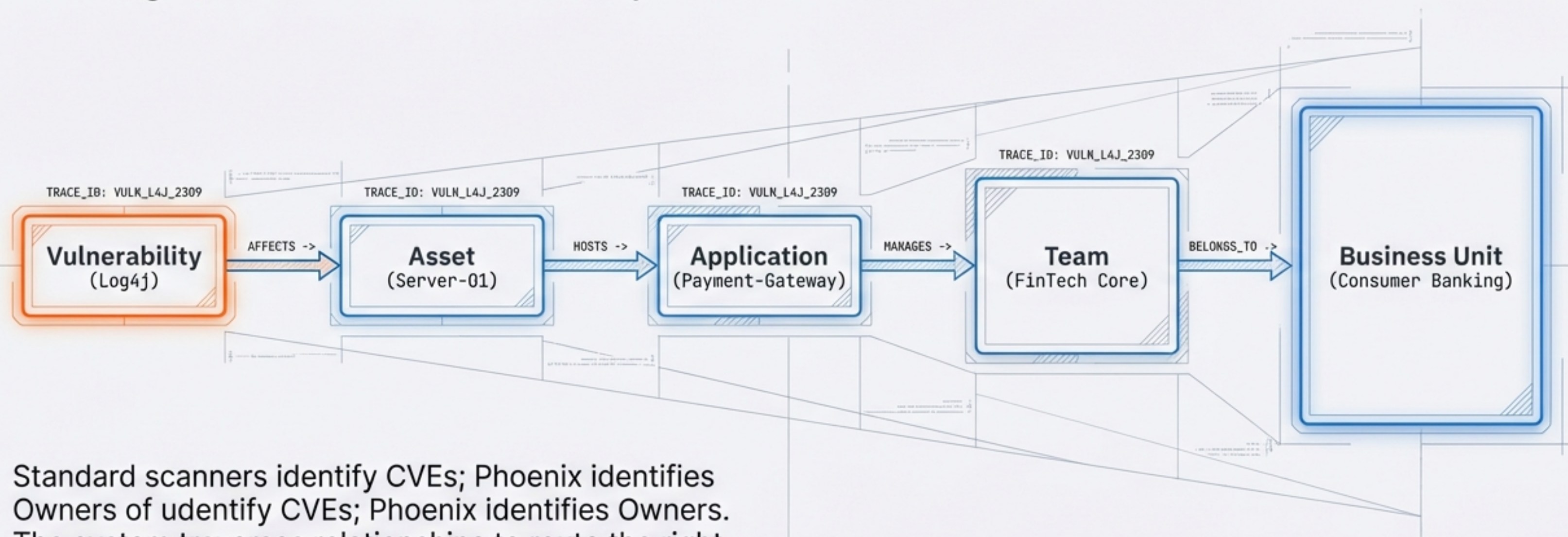


Traceability

Maintains context across the pipeline via canary tokens.

BLAST RADIUS & ORGANISATIONAL MAPPING

Solving the 'Who owns this?' problem.

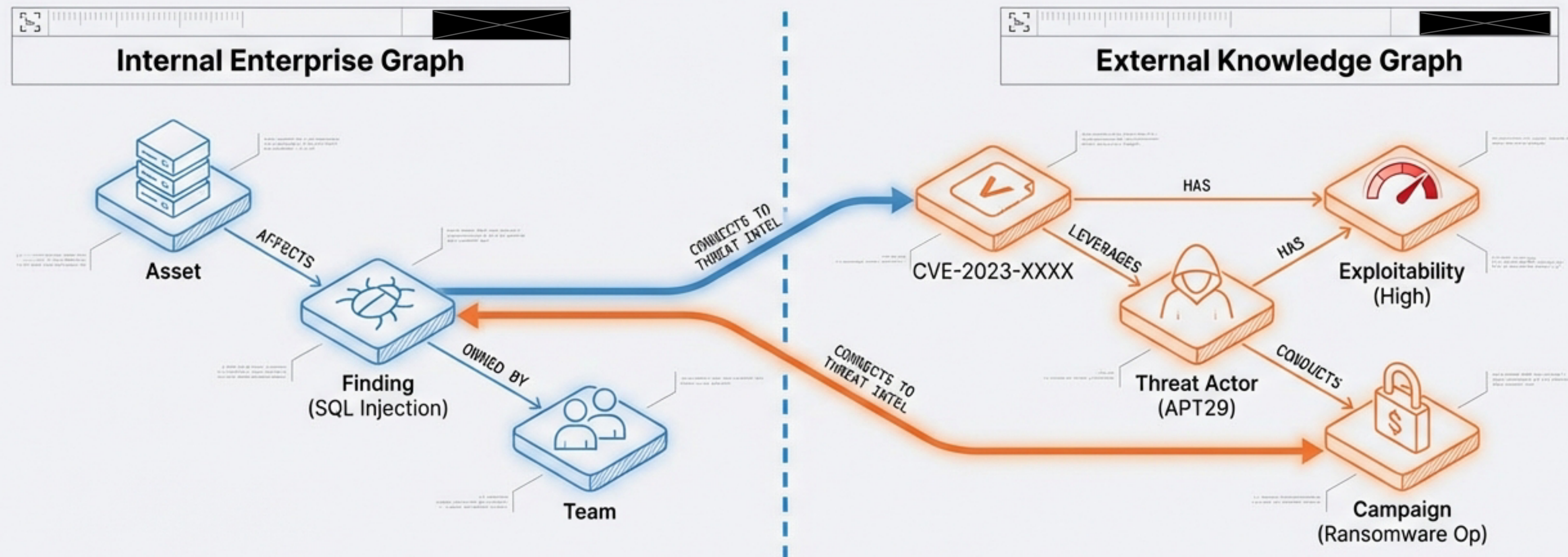


Standard scanners identify CVEs; Phoenix identifies Owners of CVEs; Phoenix identifies Owners. The system traverses relationships to route the right vulnerability to the right team automatically.

“See the vulnerabilities deployed and where to ACT.”

The Intelligence Layer: Vulnerability Intelligence Graph

Phoenix 3.30: Moving from Graph as Visualisation to Graph as Intelligence Substrate.



Connecting specific internal findings to global threat actors and active campaigns.

AGENTIC ARCHITECTURE: CONTEXT IS THE COMPASS

Building context before action prevents hallucinations.

RESEARCHER

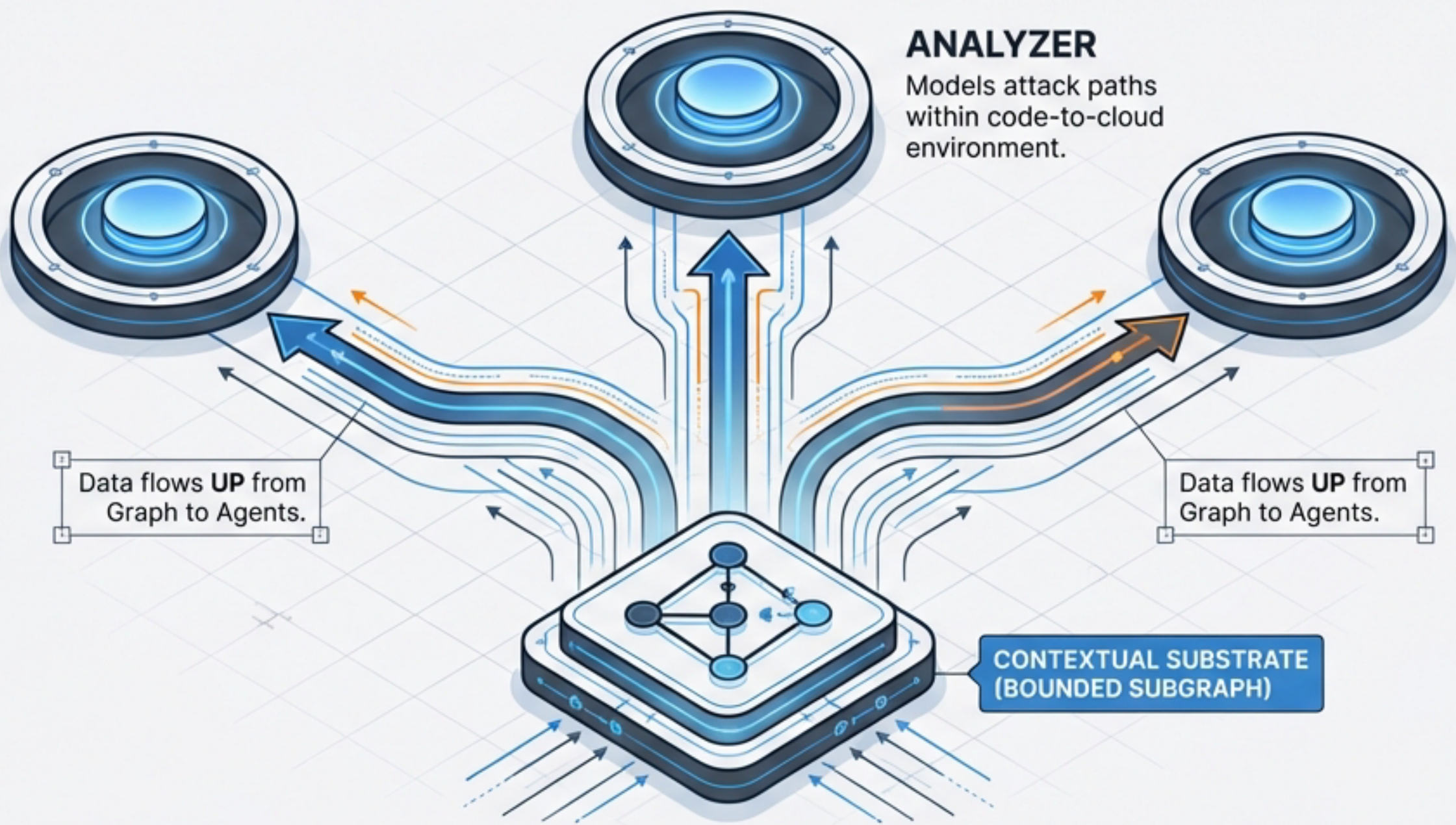
Monitors threat intel, maps vulnerabilities to actors.

ANALYZER

Models attack paths within code-to-cloud environment.

REMEDIATOR

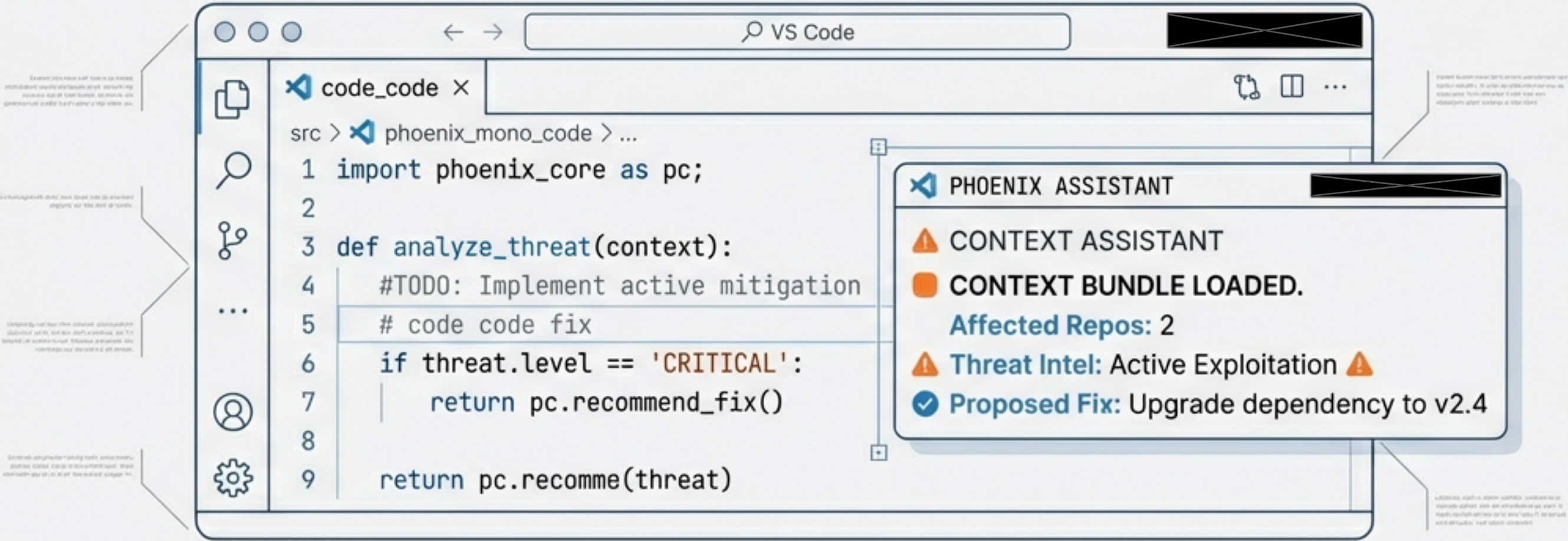
Generates environment-specific action plans.



Context is built first, then Agents act.
This prevents hallucinations.

THE DEVELOPER EXPERIENCE: IDES & 'VIBE CODING'

Phoenix 3.30: Bringing intelligence directly into the coding environment.



STRATEGY

Moving intelligence to the point of creation.



TECHNOLOGY

Model Context Protocol (MCP) exposes graph query endpoints as tools.



INTEGRATIONS

Cursor, VS Code, GitHub Copilot.



WORKFLOW

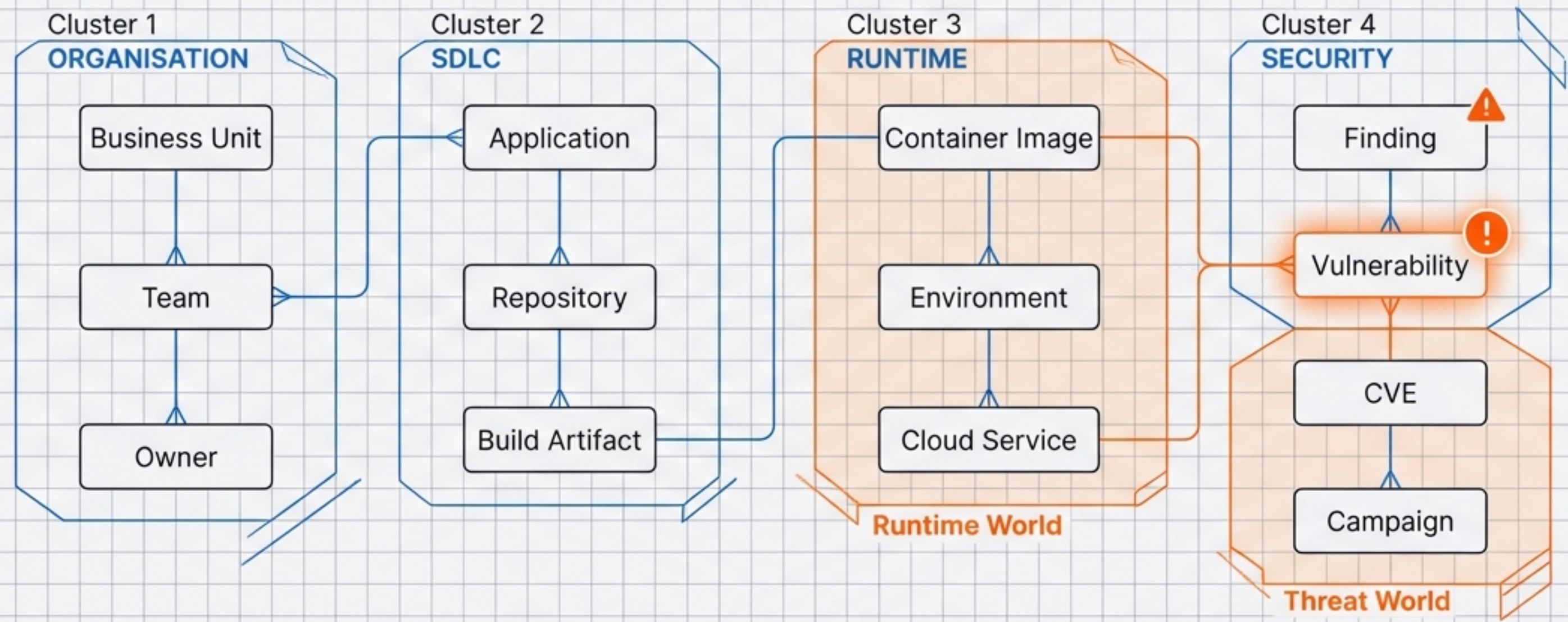
Developer asks question → Phoenix Graph supplies Context Bundle → Agent proposes fix.



TECHNICAL BRIEFING
DISPLAY

DOC ID: DEV-EXP-IDE-V1.0
DATE: 2824-18-27

INFERRED ARCHITECTURE: THE LOGICAL SCHEMA

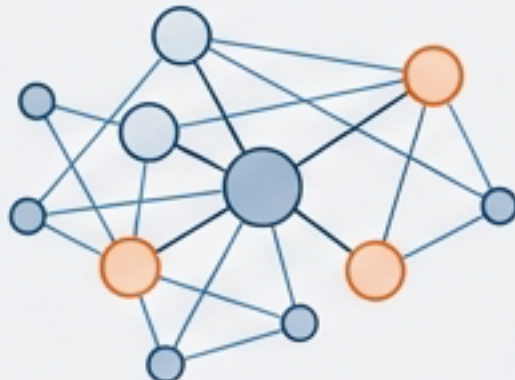
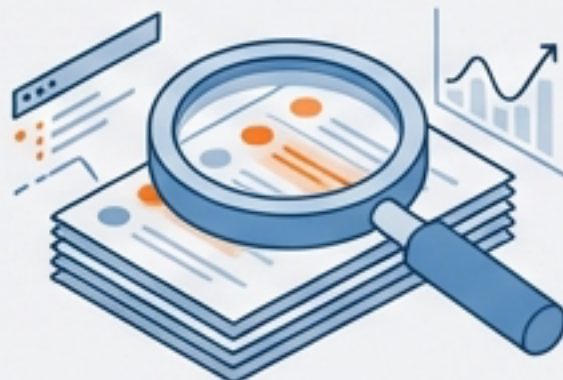


Observation
The diagram validates that Phoenix treats '**Vulnerability**' as a connector between the '**Runtime World**' and the '**Threat World**'.



IMPLEMENTATION SCENARIOS & TRADE-OFFS

Evaluating the undisclosed database engine.

 SCENARIO A: NATIVE PROPERTY GRAPH	 SCENARIO B: RELATIONAL + GRAPH PROJECTION	 SCENARIO C: SEARCH ENGINE + OVERLAYS
		
Examples: Neo4j, Neptune	Examples: PostgreSQL, SQL Server	Examples: Elasticsearch, OpenSearch
 PROS: [Graph Traversal Efficiency] Natural fit for blast radius and deep traversals.  CONS: [Aggregation Complexity] Complex analytics and time-series reporting.	 PROS: [ACID & SQL Compliance] Strong for reporting and transactional updates.  CONS: [Performance at Scale] Traversal queries expensive at scale.	 PROS: [Text Indexing Speed] Fast text search for Threat Intel.  CONS: [Limited Graph Capabilities] Weak on multi-hop path logic.

Bottom Analysis [Architectural Recommendation]
The 'Self-Healing' and 'Path-Centric' features strongly suggest a **Native** or **Hybrid Graph** approach.



TECHNICAL
BRIEFING DISPLAY

DOC ID: IMPL-SCENARIOS-V1.0
DATE: 2024-18-27

CRITICAL VALIDATION QUESTIONS

Due Diligence Checklist



Persistence Strategy: What is the authoritative store for the connected model? (Graph-native vs. Relational)



Schema Governance: How is the schema versioned as the model expands from 'Risk' to 'Intelligence'?



Deduplication Logic: How is correctness maintained across complex lineage (`Repo` → `Image` → `Runtime`)?

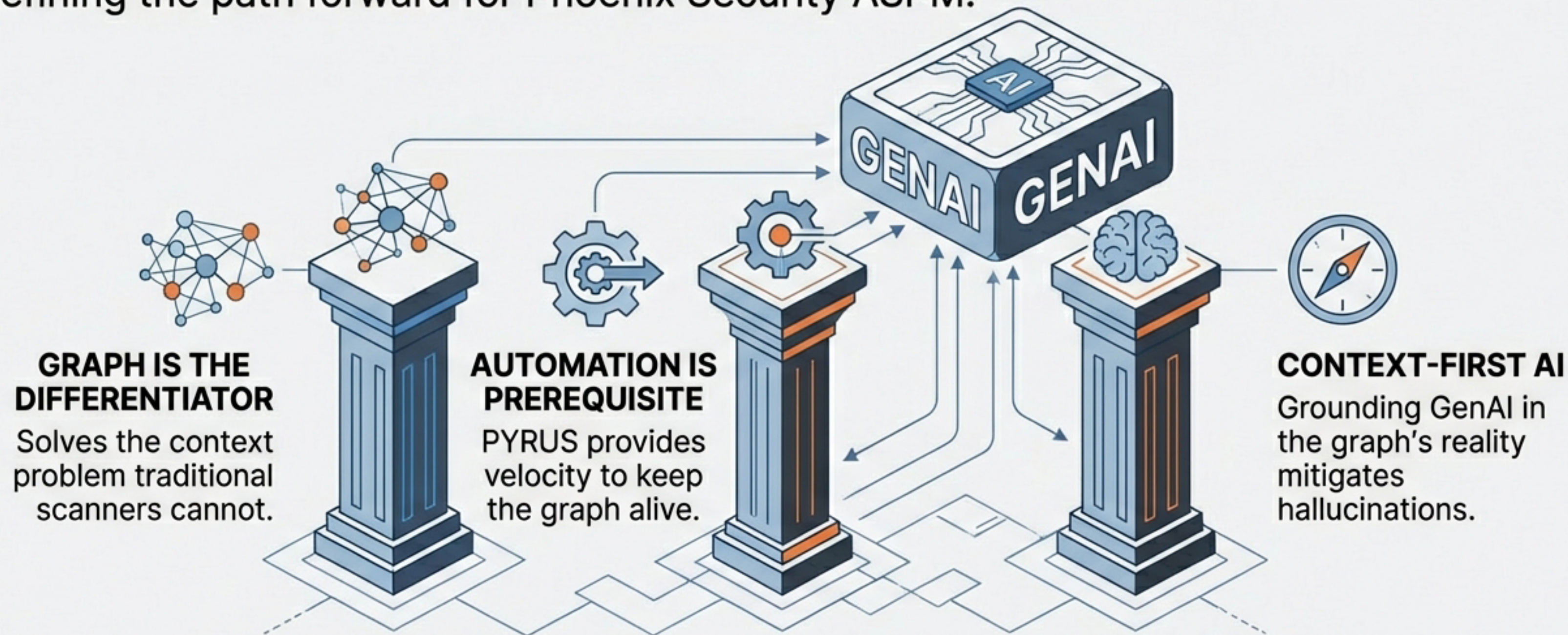


Agent Context Scoping: What is the contract for generating "bounded" context bundles? How is sensitive internal architecture redacted before being sent to an LLM?



SUMMARY & STRATEGIC OUTLOOK

Defining the path forward for Phoenix Security ASPM.



Phoenix Security positions itself as a 'Graph-First' ASPM.
Context is the Compass.



TECHNICAL
BRIEFING DISPLAY

DOC ID: SUMM-STRAT-V1.0
DATE: 2024-10-27



Confidential | Internal Review

Next Steps:

- ✓ Schedule Deep Dive Architecture Review
- ✓ POC: Validation of PYRUS Ingestion

Contact:

Architecture Team
security-architecture@internal.org